

PROGRAMACIÓN AVANZADA

Base de datos – Parte II



Ing. Matias Macrino

JDBC – JAVA DATABASE CONNECTIVITY

▶ API JDBC – Ejecución consultas SQL:

- ▶ Una vez creada la conexión, se utiliza el objeto **Statement** para enviar consultas SQL al motor de BD.
- ▶ Existen tres tipos de objetos que representan una sentencia SQL:
 - ▶ **Statement:** Es utilizado para ejecutar una sentencia SQL simple sin parámetros, devolviendo resultados.
 - ▶ **PreparedStatement:** Es utilizado para ejecutar sentencias SQL precompiladas con o sin parámetros de entrada. También suele utilizarse para la ejecución de sentencias SQL de uso frecuente.
 - ▶ **CallableStatement:** Se utiliza para ejecutar una llamada a un procedimiento almacenado de una base de datos, que puede tener parámetros de entrada, de salida y entrada/salida.

JDBC – JAVA DATABASE CONNECTIVITY

▶ API JDBC – Ejecución consultas SQL:

- ▶ Las consultas SQL se manejan a través de un objeto que implementa la interfaz **Statement**, cuya creación se realiza mediante el método **createStatement()** de la interfaz **Connection**
- ▶ Creación de un objeto Statement:
 - ✓ **Statement st = objConnection.createStatement();**
 - ✓ Siendo la variable **objConnection** del tipo **Connection**.
 - ✓ Esta operación puede provocar una *SQLException*.

JDBC – JAVA DATABASE CONNECTIVITY

▶ API JDBC – Ejecución consultas SQL:

▶ Métodos para ejecutar distintas consultas SQL:

- ▶ boolean **execute(String SQL)**: Ejecuta la sentencia SQL que se pasa por parámetro. Si la sentencia es un INSERT, UPDATE o DELETE el método devolverá false indicando que no se generan resultados. En caso de ser un SELECT el método devolverá true.
 - ▶ ResultSet **executeQuery(String SQL)**: Ejecuta una sentencia SQL de consulta que devuelve un conjunto de resultados, representado por el objeto ResultSet, para ser manipulados con éste.
 - ▶ Int **executeUpdate(String SQL)**: Ejecuta una sentencia SQL de acción que se pasa por parámetro. El método retorna el numero de registros afectados por la acción realizada.
- ▶ En el caso de producirse algún tipo de error, los tres métodos generan una excepción del tipo **SQLException**.

JDBC – JAVA DATABASE CONNECTIVITY

► API JDBC – Ejecución consultas SQL:

- Los objetos **Statement** los cerrara automáticamente el **garbage collector** (colector de basura) de Java.
- Mas allá de esto se recomienda cerrar explícitamente los objetos **Statement**. Así liberando los recursos consumidos por este objeto y también ayudando a evitar problemas potenciales de memoria de la **JVM**.
- Para poder cerrar las sentencias se deberá de utilizar el método **close()**.
- Tener presente que siempre se debe cerrar primero el objeto **Statement** y luego el objeto **Connection**, nunca al revés, ya que esto provocara una Excepción.

JDBC – JAVA DATABASE CONNECTIVITY

► API JDBC – Ejecución consultas SQL:

```
Connection con = null;
Statement stm = null;
try {
    Class.forName("com.microsoft.sqlserver.jdbc.SQLServerDriver");
} catch (ClassNotFoundException e) {
    e.printStackTrace();
}

try {
    con = DriverManager.getConnection("jdbc:sqlserver://127.0.0.1:1433;databaseName=Facos", "sa", "123456");
} catch (SQLException e) {
    e.printStackTrace();
}

try {
    stm = con.createStatement();
    boolean b = stm.execute("SELECT * FROM Clientes");
    System.out.println("Resultado SELECT: " + b);
    b = stm.execute("UPDATE Clientes SET clie_razon = 'EVENTOS PILAR SA' WHERE clie_id = 2");
    System.out.println("Resultado UPDATE: " + b);
} catch (SQLException e) {
    e.printStackTrace();
}

try {
    stm.close();
    con.close();
} catch (SQLException e) {
    e.printStackTrace();
}
```

```
Resultado SELECT: true
Resultado UPDATE: false
```

JDBC – JAVA DATABASE CONNECTIVITY

► API JDBC – Ejecución consultas SQL:

```
try{
    stm = con.createStatement();
    boolean b = stm.execute("SELECT * FROM Clientes");
    System.out.println("Resultado SELECT: " + b);
    b = stm.execute("UPDATE Clientes SET clie_razon = 'EVENTOS PILAR SA' WHERE clie_id = 2");
    System.out.println("Resultado UPDATE: " + b);
    int a = stm.executeUpdate("UPDATE Clientes SET clie_razon = 'TODOVENT S.A.' WHERE clie_id = 2");
    System.out.println("Filas Afectadas: " + a);
} catch (SQLException e){
    e.printStackTrace();
}

try {
    stm.close();
    con.close();
} catch (SQLException e){
    e.printStackTrace();
}
```

```
Resultado SELECT: true
Resultado UPDATE: false
Filas Afectadas: 1
```

JDBC – JAVA DATABASE CONNECTIVITY

► API JDBC – Ejecución consultas SQL:

	clie_id	clie_razon	clie_dom	clie_loc	prov_id	zona_id	clie_iva
1	2	TODOVENT S.A.	Ayacucho 325	Parana	SFE	CEN	RI
2	3	LA YAPA	Riobamba 1325 5º Piso	Rio Gallegos	SCR	SUR	RI
3	4	LOS_HERMANOS S.R.L.	Junin 122	Bahia Blanca	BAS	SUR	RNI
4	5	VENDEMAS	San Martin 820	Calafate	SCR	SUR	RI
5	7	QUITAPENAS S.A.	Belgrano 725	Rosario	SFE	CEN	RI

Resultado SELECT: true
Resultado UPDATE: false

	clie_id	clie_razon
1	2	EVENTOS PILAR SA
2	3	LA YAPA
3	4	LOS_HERMANOS S.R.L.
4	5	VENDEMAS
5	7	QUITAPENAS S.A.

```
try{
    stm = con.createStatement();
    boolean b = stm.execute("SELECT * FROM Clientes");
    System.out.println("Resultado SELECT: " + b);
    b = stm.execute("UPDATE Clientes SET clie_razon = 'EVENTOS PILAR SA' WHERE clie_id = 2");
    System.out.println("Resultado UPDATE: " + b);
    Thread.sleep(9000);
    int a = stm.executeUpdate("UPDATE Clientes SET clie_razon = 'TODOVENT S.A.' WHERE clie_id = 2");
    System.out.println("Filas Afectadas: " + a);
}catch(SQLException e){
    e.printStackTrace();
}
```

Resultado SELECT: true
Resultado UPDATE: false
Filas Afectadas: 1

	clie_id	clie_razon
1	2	TODOVENT S.A.
2	3	LA YAPA
3	4	LOS_HERMANOS S.R.L.
4	5	VENDEMAS
5	7	QUITAPENAS S.A.

JDBC – JAVA DATABASE CONNECTIVITY

▶ API JDBC – Manejo de los resultados:

- ▶ Se almacenan en un objeto **ResultSet** todos los resultados de la ejecución de la sentencia SQL.
- ▶ Un objeto **ResultSet** se crea al invocar el método **executeQuery()** del objeto **Statement**.
- ▶ Por defecto este objeto posee características de ser de **solo avance y solo lectura**. Lo que implica recorrer siempre desde el primer registro hacia adelante y los contenidos no son modificables.
- ▶ Para recorrer dicho objeto, encontramos el método “**next()**”. Por defecto el cursor se encuentra situado en la posición antes del primer registro, luego lo desplaza al siguiente registro retornando true si existe o false si salió del conjunto.

JDBC – JAVA DATABASE CONNECTIVITY

▶ API JDBC – Manejo de los resultados:

▶ **Acceso a los registros:**

- ▶ Existe un método `getXXX()` y el mismo tiene su propia sobrecarga. Donde XXX puede ser cualquier tipo básico de Java. Los mismos son **`getXXX(int posicion)`** o **`getXXX(String nombre_columna)`**
Ejemplos:
 - ❑ `getString("column_1");`
 - ❑ `getString(1);`
 - ❑ `getInt("column_2");`
 - ❑ `getInt(2);`
- ▶ El cursor es valido hasta que el objeto **`ResultSet`** o su objeto padre **`Statement`** es cerrado con el método **`close()`**
- ▶ Todos los métodos de la interfaz **`ResultSet`** pueden lanzar una excepción **`SQLException`**.
- ▶ Para el objeto **`ResultSet`** el índice de la primer posición es siempre 1, no 0 como en los array.

JDBC – JAVA DATABASE CONNECTIVITY

- ▶ API JDBC – Manejo de los resultados:

- ▶ **Acceso a los registros – otros metodos:**

- ▶ ***boolean isFirst()***: retorna true si el cursos apunta al primer registro.
 - ▶ ***boolean isBeforeFirst()***: retorna true si el cursor esta situado antes del primer registro.
 - ▶ ***boolean isLast()***: retorna true si el cursor esta apuntando al ultimo registro.
 - ▶ ***boolean isAfterLast()***: retorna true si el cursor se encuentra después del ultimo registro.
 - ▶ ***Int getRow()***: retorna la posición del registro actual, siendo “1” la posición del primer registro.

JDBC – JAVA DATABASE CONNECTIVITY

- ▶ API JDBC – Manejo de los resultados:

- ▶ **Cierre del ResultSet:**

- ▶ Como el resto de los objetos a utilizar el método para su cierre es **close()**.
 - ▶ Si un mismo objeto **Statement** se utiliza para crear un segundo **ResultSet**, el primer **ResultSet** será cerrado automáticamente de manera implícita.
 - ▶ Si el objetivo es tener dos **ResultSet** diferentes, para ello debemos tener abierto al mismo tiempo dos **Statement** diferentes.

JDBC – JAVA DATABASE CONNECTIVITY

► API JDBC – Manejo de los resultados:

► **Información sobre los datos:**

- Además de obtener los datos, **JDBC** nos proporciona una interfaz **ResultSetMetaData**. Esta misma nos permite obtener información sobre las características de los datos referenciados al **ResultSet**.
- Con el método **getMetaData()** podremos generar un objeto **ResultSetMetaData**.
- Metodos:
 - **int getColumnCount()**: retorna el numero de columnas del conjunto de registros referidos al **ResultSet**.
 - **String getColumnName(int posicion)**: retorna el nombre de una columna a partir de su posición.
 - **int getColumnType(int posicion)**: retorna una constante entera que representa el tipo de dato SQL. Los cuales se encuentran definido en **java.sql.Types**
 - **String getColumnName(int posición)**: retorna el nombre del tipo de dato soportado por el campo, según esta definido en el motor de base de datos.

JDBC – JAVA DATABASE CONNECTIVITY

▶ API JDBC – Manejo de los resultados:

▶ **Modificar ResultSet:**

- ▶ Para cambiar el comportamiento por default de nuestro objeto **ResultSet**, lo podremos hacer configurando nuestro objeto **Statement**, utilizando su constructor.

□ Sobrecarga de createStatement():

- ✓ **Statement st = objConnection.createStatement(var1, var2);**
- ✓ Var1 □ indica el tipo de objeto **ResultSet** a crear.
- ✓ Var2 □ indica si el **ResultSet** es solo de escritura o si admite modificaciones.

JDBC – JAVA DATABASE CONNECTIVITY

- ▶ API JDBC – Ejecución consultas SQL:

- ▶ Las constantes del primer parámetro son:

- ▶ TYPE_FORWARD_ONLY (opción por defecto)

- ▶ Se crea un objeto **ResultSet** con movimiento únicamente hacia delante (forward-only).

- ▶ TYPE_SCROLL_INSENSITIVE

- ▶ Se crea un objeto **ResultSet** que permite todo tipo de movimientos. Pero mientras está abierto, no será consciente de los cambios que se realicen sobre los datos que está mostrando, y por lo tanto no mostrará estas modificaciones.

- ▶ TYPE_SCROLL_SENSITIVE

- ▶ Al igual que el anterior permite todo tipo de movimientos, y además permite ver los cambios que se realizan sobre los datos que contiene.

JDBC – JAVA DATABASE CONNECTIVITY

- ▶ API JDBC – Ejecución consultas SQL:

- ▶ Las constantes del segundo parámetro son:

- ▶ CONCUR_READ_ONLY (opción por defecto)

- ▶ Indica que el **ResultSet** es sólo de lectura.

- ▶ CONCUR_UPDATABLE

- ▶ Permite realizar modificaciones sobre los datos que contiene el **ResultSet**